

Bus tracker — hardware integration spec

For the ESP32 + GPS + cellular tracker units fitted to the six campus buses.

Plain MQTT over TCP. No TLS, no certificates, no HTTP. The unit publishes to a public MQTT broker. Because the link is unencrypted and the broker is open, each unit signs its data with a secret that is never transmitted — an eavesdropper can read positions but cannot forge one.

> **Change from the previous revision.** Units used to POST to a relay at `reverse.ibinujaleel.dev:8081`, which forwarded to a database. That relay and its VM are gone. The **payload and the signature are unchanged**; what changes is that it is now published to an MQTT topic instead of POSTed, and the signature travels in the message body rather than an HTTP header.

1. What the device has to do

In one sentence: **get a GPS fix, sign it, publish it every 5 seconds.**

```
on boot:
  connect to broker
  publish "online" to the status topic (retained)
  register Last Will: "offline" on the same topic (retained)

loop:
  read GPS
  build JSON body (with an ever-increasing counter)
  sig = HMAC_SHA256(device_secret, body)
  publish sig + "." + body to the gps topic
  wait 5 s
```

No route logic, no map data, no stop coordinates, no TLS. The server works out which route the bus is on, whether it is parked, and which way it is going.

2. Broker and topics

Broker	broker.emqx.io
Port	1883, plain TCP, no TLS
Protocol	MQTT 3.1.1
Username / password	none
Clean session	yes
Keepalive	60 s (send PINGREQ)
Client id	unique per unit, e.g. cbt-esp32-01

<ROOT> below is `cbt7f3c9e21b` — a fixed, unguessable prefix so the buses do not collide with the thousands of other users of this public broker.

It must match the server's `BUS_TOPIC_ROOT` exactly.

Topic	When	QoS	Retained
<ROOT>/bus/<busid>/gps	every fix	0	no
<ROOT>/bus/<busid>/status	on connect: online	1	yes
<ROOT>/bus/<busid>/status	Last Will: offline	1	yes

Positions are QoS 0 on purpose: a fix that needs retrying is already too old to be worth delivering. Never buffer and replay old fixes.

The Last Will is what tells students a bus has dropped off. Register it in the CONNECT packet — the broker publishes it for you when the connection dies.

3. The message

```
d0d9d4c131e5b25c618be70720948ae790b564804543d6f8a7
43a88716567e39.{"b":"bus1","d":"esp32-01","c":1234,
"lat":11.3185,"lng":75.9379,"spd":6.4,"hdg":312}

(one line on the wire -- wrapped here only to fit the page)
```

That is: **64 hex characters, a dot, then the exact JSON bytes you signed.**

The server splits on the first dot and verifies the remainder.

Field	Type	Meaning
b	string	Bus id. Fixed per unit — see §5.
d	string	Device id. Fixed per unit — see §5.
c	integer	Counter. Must increase on every message, forever. See §6.
lat / lng	number	Decimal degrees, WGS84. 6 decimals is plenty.
spd	number	Metres per second. NMEA gives knots — multiply by 0.514444.
hdg	number	Degrees 0–360, 0 = north. Send null if unknown — never 0.

Do not send a timestamp. The server stamps arrival time itself, so a wrong RTC cannot make a live bus look offline.

There is no reply. MQTT QoS 0 is fire-and-forget, so the unit cannot tell whether a fix was accepted — which is fine, because the next one is along in five seconds. Rejection reasons appear in the server log, not on the device.

Why the signature is inside the payload: MQTT has no header to put it in.

4. Why these rejections happen

The server drops a message silently in these cases. If a bus never appears, check them in order:

Cause	Fix
Malformed frame	Must be exactly 64 hex chars, then ., then the JSON
Bad signature	Sign the exact bytes you publish — see §6
Unknown device	d is not in the server's device list
Wrong bus	This device is not bound to that b, or published on another bus's topic
Replay	c was not higher than the last accepted value
Out of area	Coordinates outside the campus bounding box
Implausible jump	Position moved faster than 25 m/s since the last fix

5. Per-unit identity

Each unit is flashed with its own id, bus and secret. **A unit gets only its own secret.**

These are issued separately — they are not in this document.

Device id	Bus	Secret
esp32-01	bus1	issued separately
esp32-02	bus2	issued separately
esp32-03	bus3	issued separately
esp32-04	bus4	issued separately
esp32-05	ac_mbh	issued separately
esp32-06	ac_lh	issued separately

The secret is a 32-character hex string. Store it in NVS, not in a source file that ends up on GitHub.

6. Signing — the important part

```
sig      = hex( HMAC_SHA256( device_secret, exact_body_bytes ) )
message = sig + "." + body
```

Sign the **exact bytes you publish**. Build the JSON string once, sign that string, publish that same string after the dot. Re-serialising between signing and publishing is the usual cause of a rejected signature — a different key order or an extra space changes the hash completely.

The secret is never sent. The broker is public, so anyone can read bus positions and anyone can publish to these topics — but without the secret they cannot produce a message the server will accept.

This signature is the only thing protecting the system.

Treat the secret accordingly.

The counter

The counter `c` must be strictly greater than the previous accepted value, forever, per device. It stops someone from recording a valid request and replaying it later.

Store it in NVS and increment on every send. **Persist across reboot** — if it resets to 0 the server rejects everything as a replay until it climbs past the old value, and because MQTT gives no reply the unit will look fine while publishing into the void. A simple, safe scheme:

```
c = unix_seconds_from_GPS // monotonic, survives reboot for free
```

If you have no reliable clock, use an NVS counter and bump it by 100 on boot to cover any unwritten increments.

ESP32 notes

`MBEDTLS_MD_HMAC()` with `MBEDTLS_MD_SHA256`, or Arduino's `MBEDTLS` wrapper.

The ESP32 has SHA-256 in hardware; signing takes microseconds and adds 65 bytes to each message. Do not roll your own HMAC.

MQTT client. The SIM900A has no MQTT AT commands, so the client is built on the ESP32 over the modem's TCP socket (`AT+CIPSTART`). You need only three packet types — CONNECT (with the Will), PUBLISH at QoS 0, and PINGREQ — which is roughly 100 lines. Alternatively run the modem in PPP mode and use `esp-mqtt` with a plain `mqtt://` URI. **No TLS either way**, which is what makes this practical on 2G.

7. Constraints

7.1 Interval: 5 seconds

Not arbitrary. Stop-arrival detection needs at least two fixes inside a 40 m radius; at ~25 km/h a bus crosses that in ~11 s. At 10 s intervals you get one fix and the app starts missing arrivals. **Do not go slower than 5 s without telling us.**

7.2 Keep one connection open

MQTT is a session: connect once and hold it, sending `PINGREQ` every 60 s.

Reconnecting per fix multiplies data use and power draw for no benefit, and loses the Last Will that tells students the bus dropped off.

On reconnect, re-publish `online` to the status topic.

7.3 Data budget

~200 bytes per message (100 payload + 65 signature + MQTT and TCP overhead) over a 10.9-hour service day:

Interval	Per bus/day	Per bus/month	All 6/month
5 s	1.6 MB	48 MB	0.3 GB
10 s	0.8 MB	24 MB	0.15 GB

A 500 MB/month SIM is comfortable. MQTT roughly halves this against the old HTTP POST, which re-sent method, path and headers on every fix.

7.4 Do NOT buffer when offline

Campus has dead spots. **Drop fixes you could not send.** Do not queue them.

A position is only useful while it is current: the server stamps arrival time itself, so a fix flushed from a buffer three minutes later is indistinguishable from a live one and would teleport the bus across the map. Freshness beats completeness.

Never block the GPS loop on a network call.

7.5 Power

Buses cut power with the ignition; expect 12–24 V, dirty, with load-dump spikes. Use an automotive-rated buck converter, and a supercapacitor or small LiPo so the unit can finish its current request and shut down cleanly.

7.6 Module choice

Since there is no TLS, almost any module works.

SIM900A and SIM800L are fine here, as are SIM7600 / A7670 / SIM7670. Requirements: a TCP client that can hold an open socket, and ~1 KB buffers. No modem-side MQTT or TLS support is needed: the MQTT client lives on the ESP32.

8. Bring-up checklist

MQTT gives no acknowledgement at QoS 0, so verify by watching the broker rather than the device. Subscribe from a laptop first and leave it running:

```
mosquitto_sub -h broker.emqx.io -t 'cbt7f3c9e21b/bus/+/#' -v
```

On the bench, before fitting anything to a bus:

1. Unit connects to the broker — `online` appears on the status topic.
2. One signed publish appears on the `gps` topic, correctly framed.
3. Ask us to confirm the server **accepted** it (it appears on `cbt7f3c9e21b/live/buses`). A message on the `gps` topic only proves it was published, not that the signature passed.
4. Corrupt one byte of the signature → server logs `bad signature`, bus does not move.
5. Power off the unit → `offline` appears on the status topic within ~90 s.

Proves the Last Will is registered.

6. Power-cycle → resumes with no manual step, and the counter does not restart.
7. Pull the antenna → reconnects on return; no lock-up, no reboot loop.
8. Run 30 minutes → stable memory, connection still alive, no reconnect storm.

Reference message (replace the secret, then compare against your firmware's

bytes — this is the fastest way to find a signing bug):

```
BODY='{ "b": "bus1", "d": "esp32-01", "c": 1, "lat": 11.3185, '
BODY=$BODY'"lng": 75.9379, "spd": 6.4, "hdg": 312}'
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "YOUR_SECRET_HERE" -hex | awk '{print $2}')
mosquitto_pub -h broker.emqx.io -t 'cbt7f3c9e21b/bus/bus1/gps' -m "$SIG.$BODY"
```

With YOUR_SECRET_HERE as the secret, that signature is exactly:

```
d0d9d4c131e5b25c618be70720948ae790b564804543d6f8a743a88716567e39
```

If your firmware produces a different value for that body and secret, the bug

is in your HMAC or your JSON serialisation — not in the server.

9. Please do NOT

- Send speed in km/h or knots. **Metres per second.**
- Send `hdg`: `0` when heading is unknown — send `null`. Zero means north and will point every parked bus the same wrong way on the map.
- Reset the counter on reboot. Everything will be silently rejected until it catches up, and MQTT will not tell you.
- Re-serialise the JSON after signing it. Sign and publish the same bytes.
- Buffer fixes while offline and flush them later. Drop them.
- Share a secret or a device id between units.
- Publish on another bus's topic. The server checks the binding both ways.
- Invent bus ids. Only the six in §4 exist.
- Commit secrets to a public repo.

Questions come to us rather than being guessed at — a wrong assumption here

costs a site visit to six buses.